Modèles algorithmiques fondamentaux sur structures à accès direct (Tableaux et Chaînes en C++)

Thème principal : Schémas réutilisables de parcours : traitement systématique, traitement conditionné et recherche de première occurrence

Synthèse exécutive : Ce cours pose les fondations de la programmation structurée en R1.01 en formalisant les patrons de conception algorithmiques (design patterns algorithmiques) élémentaires sur les structures à accès direct. Une structure à accès direct (comme un tableau 'std::vector' ou une chaîne 'std::string') permet d'accéder à n'importe quel élément en temps constant O(1) grâce à son indice. On y distingue deux grandes familles de parcours : le parcours complet (exécuté avec une boucle 'for' quand le nombre d'itérations est connu à l'avance) et le parcours partiel/recherche (exécuté avec une boucle 'while' pour permettre un arrêt précoce dès que le composant recherché est identifié). La maîtrise de ces schémas garantit l'écriture d'un code correct, lisible, exempt de dépassements de mémoire et conforme aux exigences strictes du BUT Informatique.

1. Définition et propriétés des structures à accès direct

Une structure de données est dite 'à accès direct' lorsqu'il existe une fonction de repérage directe entre un indice entier et l'emplacement mémoire de la donnée. En C++, la notation 't[i]' calcule l'adresse 'adresse_base + i * taille_type'. L'accès se fait donc en temps constant O(1). Cependant, cette liberté impose une rigueur absolue : toute tentative d'accès à l'indice 'N' (alors que la taille est 'N') provoque un comportement indéfini (Undefined Behavior) ou un crash 'Segmentation Fault'. Les algorithmes rédigés sur ces structures doivent impérativement borner les indices strictement au domaine [0, N-1].

- Accès direct : capacité d'atteindre le i-ème élément directement par son adresse mémoire indexée sans parcourir les éléments précédents.
- Indexation : en C++, les indices commencent obligatoirement à 0 et se terminent à la taille N-1.
- Structures concernées : tableaux statiques, std::vector, std::string.

Cas concret enseignant : *L'enseignant illustre cela avec l'analogie d'un meuble à tiroirs numérotés : si le meuble a 10 tiroirs (numérotés de 0 à 9), ouvrir le tiroir n°5 ne demande pas d'ouvrir les tiroirs 0 à 4. En revanche, tenter d'ouvrir le tiroir 10 revient à essayer de tirer une poignée qui n'existe pas et casser le meuble.*

2. Modèle 1 : Parcours complet avec traitement systématique

Le parcours complet systématique s'applique lorsqu'une opération doit être appliquée sans exception à l'ensemble des éléments de la structure. Comme le nombre de répétitions est déterminé à l'avance (exactement égal à la taille N de la structure), la boucle 'for' est la seule structure de contrôle académiquement valide. Le schéma algorithmique standard s'écrit : for (unsigned int i = 0; i < t.size(); ++i) { // Traitement de t[i] }

- Invariant : chaque élément du tableau est visité exactement une fois.
- Structure de contrôle : boucle pour (for).
- Usage : calcul de somme, moyenne, duplication, affichage exhaustif.

Cas concret enseignant : *Le professeur donne l'exemple du calcul du prix TTC pour l'ensemble des articles d'un panier d'achat : chaque produit doit obligatoirement subir la même opération d'application de la TVA.*

3. Modèle 2 : Parcours complet avec traitement conditionné

Ce modèle conserve l'exhaustivité du parcours complet (utilisation d'une boucle 'for'), mais restreint le traitement à un sous-ensemble d'éléments respectant un prédicat logique P(t[i]). L'erreur fréquente chez les débutants est de tenter de sortir de la boucle lorsqu'un élément ne respecte pas le critère, ce qui fausse le résultat exhaustif. Le

schéma de référence est : for (unsigned int i = 0; i < t.size(); ++i) { if (predicat(t[i])) { // Traitement spécifique de t[i] } }

- Combinaison d'un parcours exhaustif et d'une structure conditionnelle (si/if).
- Tous les éléments sont inspectés, mais tous ne sont pas transformés ou comptabilisés.
- Usage : comptage d'éléments selon critère, filtrage, statistiques sélectives.

Cas concret enseignant : *Exemple de l'appel en classe : le professeur parcourt la liste de tous les étudiants présents dans la salle (parcours complet), mais ne coche sur sa feuille que ceux qui ont levé la main pour poser une question (traitement conditionné).*

4. Modèle 3 : Recherche de première occurrence (Parcours partiel avec arrêt)

Lorsqu'il s'agit de trouver un élément précis dans un tableau non trié, continuer à parcourir le tableau après avoir trouvé l'élément est une faute d'optimisation et de conception algorithmique. L'usage de la boucle 'for' est ici proscrit en BUT Info car le nombre d'itérations n'est pas connu à priori. On utilise une boucle 'while' combinant deux conditions d'arrêt : n'avoir pas dépassé la fin du tableau ET ne pas encore avoir trouvé l'élément. Structure canonique C++ : unsigned int i = 0; bool trouve = false; while (i < t.size() && !trouve) { if (t[i] == cible) { trouve = true; } else { ++i; } }

- Arrêt précoce : la boucle doit s'arrêter dès que la condition est remplie.
- Structure de contrôle : boucle tant que (while).
- Variables d'état : un indice de position 'i' et un booléen de contrôle (ex: 'trouve').

Cas concret enseignant : *Exemple du trousseau de clés : si vous cherchez la clé de votre voiture sur un trousseau de 10 clés et que la 3ème clé fonctionne, vous ne continuez pas à tester les 7 clés restantes.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi est-il interdit d'utiliser une boucle 'for' avec l'instruction 'break' pour stopper la recherche quand on a trouvé l'élément ?
Réponse du professeur : L'instruction 'break' crée des sorties multiples indisciplinées, rompant le principe de la programmation structurée à entrée unique et sortie unique. Cela rend les preuves d'algorithmes et le débogage complexes. L'évaluation d'examen pénalise l'usage du 'break' dans une boucle 'for' pour interrompre un parcours : utilisez une boucle 'while' avec un booléen explicite.

Question étudiant : Dans la condition du 'while (i < t.size() && !trouve)', que se passe-t-il si le tableau est vide ?
Réponse du professeur : Grâce à l'évaluation 'court-circuit' (lazy evaluation) des opérateurs logiques en C++, la première condition 'i < t.size()' (0 < 0) est évaluée à faux. Le langage n'évalue donc même pas la suite de la condition et n'essaie jamais d'accéder à 't[i]', ce qui garantit qu'aucune erreur de mémoire n'a lieu.

■ Exercice & Démonstration d'Application

Énoncé : Écrire une fonction C++ 'trouverPremierNegatif' qui prend en paramètre un 'std::vector' et retourne l'indice du premier entier strictement négatif. Si aucun entier négatif n'existe dans le tableau, la fonction doit retourner -1.

Résolution pas à pas :
1. Identifier le modèle algorithmique : il s'agit d'une recherche de première occurrence (parcours partiel avec arrêt précoce). 2. Choisir la structure de contrôle : boucle 'while'. 3. Initialiser un indice 'i = 0' et un booléen 'trouve = false'. 4. Définir la condition de garde du 'while' : '(i < v.size() && !trouve)'. 5. Dans la boucle : si 'v[i] < 0', positionner 'trouve = true'. Sinon, incrémenter 'i'. 6. Après la boucle : si 'trouve' est vrai, retourner 'static_cast(i)', sinon retourner -1. Code C++ équivalent : int trouverPremierNegatif(const std::vector& v) { unsigned int i = 0; bool trouve = false; while (i < v.size() && !trouve) { if (v[i] < 0) { trouve = true; } else { ++i; } } if (trouve) { return static_cast(i); } return -1; }

Résultat : La fonction garantit une complexité temporelle O(k) où k est l'indice du premier élément négatif (au pire O(N) si absent), n'accède jamais à une mémoire invalide et respecte scrupuleusement les normes académiques de programmation structurée.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la condition de fin stricte d'un parcours complet par indice sur un vector de taille N ?

R : L'indice i atteint la valeur N (la dernière case valide visitée étant N-1), la condition d'arrêt de la boucle étant 'i < N'.

Q : Pourquoi la boucle 'while' est-elle obligatoire pour la recherche d'occurrence par rapport à la boucle 'for' ?

R : Parce que le nombre d'itérations n'est pas connu au préalable ; la boucle doit pouvoir s'interrompre dès la découverte de l'élément recherché sans utiliser d'instruction de rupture brutale comme 'break'.

Q : Quel est le risque majeur lors de l'inversion des conditions 'while (t[i] != cible && i < t.size())' ?

R : Un dépassement de tableau (Segmentation Fault) : si l'élément n'est pas présent, 'i' atteindra la valeur 't.size()'. 't[i]' sera évalué en premier avant de tester 'i < t.size()', provoquant un accès mémoire hors limites.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765